

Integration Process and Testing

Version: 1.24

21-01-2026

Content

1	Introduction.....	5
2	Purpose of the document	5
3	Audience	5
4	Glossary	5
5	Data overview	6
5.1	Device-related data (Master-data)	6
5.2	Measurement Data	7
6	Integration Process	9
6.1	Logical architecture	9
6.2	General flow	10
6.3	Tech stack	11
6.3.1	JSON	11
6.3.2	OCI proxy	12
6.3.3	mTLS.....	12
6.3.4	RESTful API	12
6.3.5	Rabbit MQ	12
6.4	Authentication.....	13
6.4.1	Certificate request.....	13
6.4.2	Certificate testing	15
6.4.3	Authentication flow.....	16
6.5	OPTION 1: Data transfer via FEAK API	17
6.5.1	Versioning	17
6.5.2	Device-related data transfer	17
6.5.3	Measurement data transfer	20
6.5.4	Testing.....	23
6.6	OPTION 2: Queue-based data transfer.....	23

6.6.1	Flow chart	23
6.6.2	Architecural overview.....	24
6.6.3	Device-related data	25
6.6.4	Measurement data.....	29
6.6.5	Error handling.....	33
6.6.6	Extended functionality	35
7	Sandbox environment	37
7.1	Requirements from inverter manufacturer side	38
7.2	Access list.....	38
7.2.1	FEAK API host	38
7.2.2	Queue-based data transfer access points.....	38
7.2.3	PodRegistry host.....	39
7.2.4	MDAPI host.....	39
8	Support, contact.....	39

List of figures

1. figure - Logical architecture	10
2. figure - General integration flow	11
3. figure - Certificate request flow	15
4. figure - Authentication process	16
5. figure - Queue-based data transfer flow chart	24
6. figure - Logical architecture of queue-based data transfer	25

1 Introduction

FEAK is developing and operating a system that collects energy consumption, production, and storage data from the related devices across the country. The system also provides data analysis services based on the collected data. The development is supported by the Government of Hungary and the European Union under the DIMOP_Plusz-2.1.2-23-2023-00001.

The objective is to ensure that the energy consumption and production data of related devices in Hungary are integrated into the system. As devices associated with energy production, inverters are legally required to provide data to the system.

2 Purpose of the document

This document outlines the integration processes to this system and provides supporting guidelines for implementation and testing for inverter manufacturers. Its principal purpose is to identify and collect the fundamental components of the integration process and provide a clear, structured summary to simplify and optimize the integration workflow, making it more efficient and effective.

3 Audience

The content in this document is intended for software developers, QA engineers, DevOps specialists, technical managers and stakeholders who are involved in implementing, or maintaining the integration process from the inverter manufacturer's side.

4 Glossary

Term	Definition
Inverter Manufacturer	A company that designs and produces devices converting DC electricity into AC.
FEAK	Független Energetikai Adatközpont Zrt.

	Hungarian independent energy data center, responsible for collecting and storing inverter data.
HMKE	Household-Size Small Power Plant It refers to small-scale (< 50 kVA), typically residential power generation units in Hungary, usually based on renewable energy sources like solar panels.
PodRegistry	Internal component for handling all device-related data
MDAPI	Internal component for handling measurement data
Inverter Cloud Connector	Internal component, responsible for parsing queue-based messages.

5 Data overview

The inverters data managed by the FEAK system can be categorized into two distinct groups: device-related data and measurement data.

5.1 Device-related data (Master-data)

Device-related data contains information that is used to identify the device and also defines the general characteristics of the device.

- **Device serial number**
The unique identifier of the device
- **Brand**
The manufacturer of the device
- **Model**
The type and specific version of the device within the brand's product range
- **Nominal power**
The nominal AC output power refers to the peak power the inverter can continuously supply to the main grid under normal conditions.

- **AC side voltage range**

The AC output voltage range specifies the acceptable range of voltages that the solar inverter can generate for grid connection.

- **Device installation date**

The device installation date refers to the date on which the equipment was set up and became "operational"

- **Device installation address**

The physical location where the device is installed, including the following standard fields:

- **country:** Country code of the address.
- **zipCode:** Postal code of the address.
- **city:** Name of the city.
- **landRegistryNumber:** Identifier used in the land registry, also known as the parcel number.
- **street:** Name of the street.
- **streetType:** Type of the street (e.g., Street, Avenue, Boulevard). The value set is available in the attachment.
- **streetCode:** Official street code
- **streetCodeSupplement:** Additional code or suffix for the street, used for more precise identification.
- **building:** Building number or identifier.
- **stairway:** Stairway or block identifier within a building.
- **door:** Door or apartment number.
- **floor:** Floor number or level within the building.
- **latitude:** Geographic latitude coordinate of the address.
- **longitude:** Geographic longitude coordinate of the address.

5.2 Measurement Data

The data reporting requirement for manufacturers mandates the submission of the following measurement data. Measurement data from the devices is transmitted in near real-time (the minimum frequency is once per hour). The data is structured in 5-minute interval readings.

- **Generation capacity of an HMKE (kVA)**

The average value of the apparent power output (S) from the inverter, averaged over 5 minutes, expressed in kVA.

- **Average value of electricity produced by an inverter (kWh)**

Amount of electricity produced by an inverter, in kWh, 5 minutes unit.

- **Amount of energy consumed for own usage (kWh)**

From the amount of electricity generated by the HMKE (PV System), the amount of energy used for captive use at the place of use, in 5 minutes unit, in kWh

- **Voltage and current values per phase**

Average AC voltage values per phase at the AC side terminals of the inverter, in V, in 5 minutes unit

AC current values measured per phase on the AC side terminals of the inverter, in A, in 5 minutes unit

- **Power factor**

The value of the inverter power factor $\cos(\phi)$

The FEAK system is designed to handle measurement data from multiple electrical devices. A standardized approach to data management is applied using OBIS codes to ensure uniformity and interoperability.

The table below shows the mapping between OBIS codes, the corresponding measurement data, and the unit.

OBIS code	Type of measurement data
9.7.0	Generation capacity of an inverter (kVA)
2.8.0	Average value of electricity produced by an inverter (kWh)
X.1.8.0	Amount of energy consumed for own usage (kWh)
32.7.0	Voltage phase L1 (V)
52.7.0	Voltage phase L2 (V)

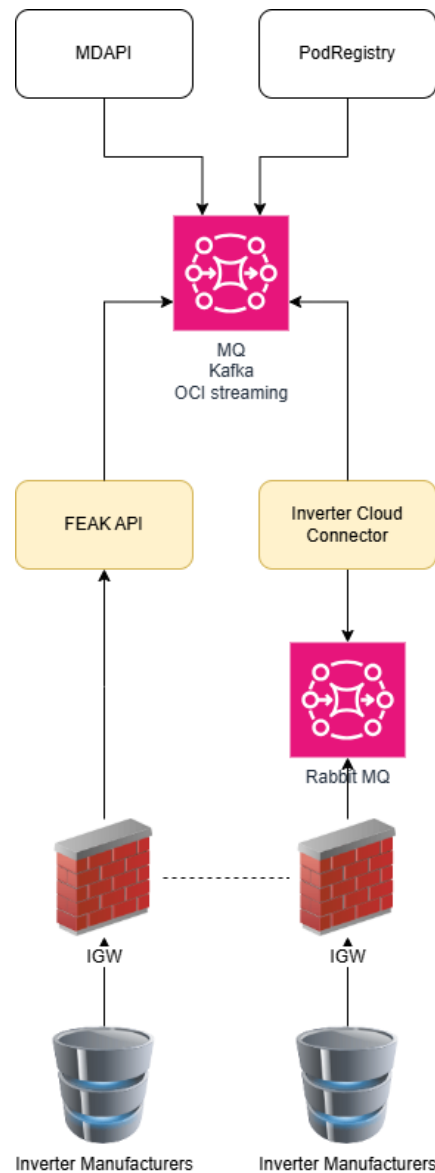
72.7.0	Voltage phase L3 (V)
31.7.0	Current phase L1 (A)
51.7.0	Current phase L2 (A)
71.7.0	Current phase L3 (A)
13.7.0	Power factor

6 Integration Process

The system supports two primary methods for data transfer: API-based data transfer or queue-based data transfer. Both methods can be used to transfer device-related and measurement data.

6.1 Logical architecture

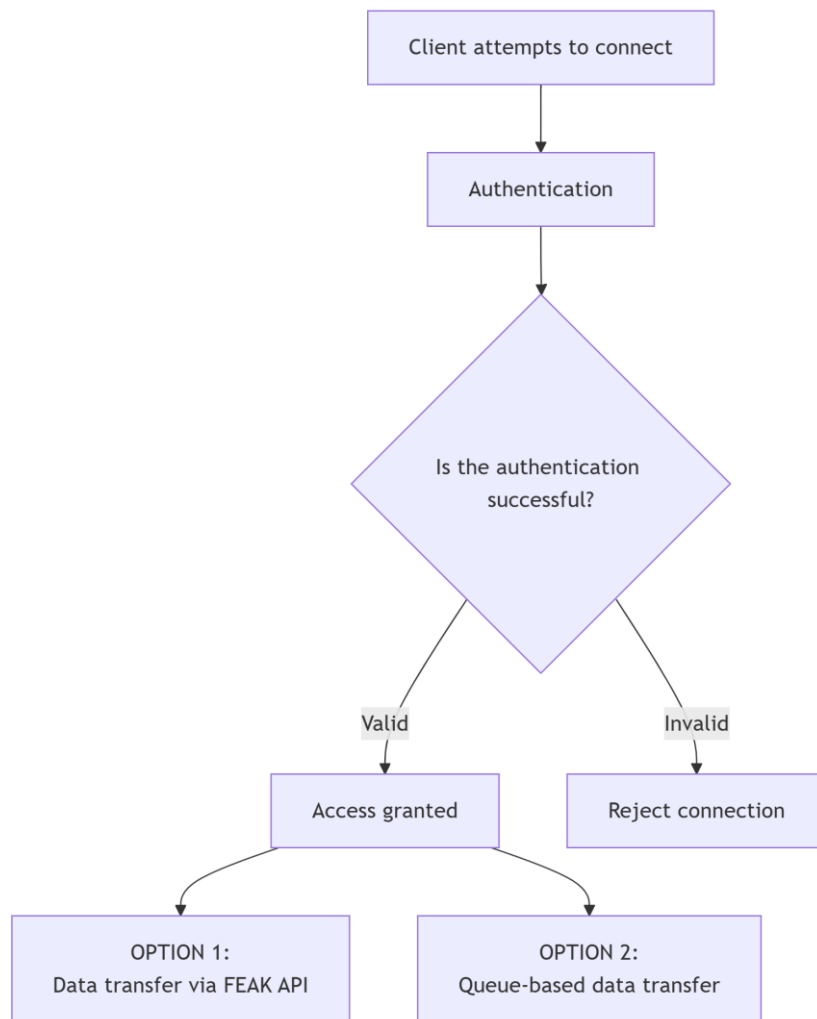
The following diagram represents the logical architecture of the system. It illustrates the main system components, and how they interact with each other within the overall solution. This logical view focuses on the structure of the system from a high-level perspective, abstracting away physical deployment details in order to emphasize data flow, functional modules, and integration points.



1. figure - Logical architecture

6.2 General flow

The following diagram applies to both device-related data and measurement data types. While certain implementation details may differ between the two, the core workflow remain consistent across both data options.



2. figure - General integration flow

In the case of device-related data, the start business event is a new inverter in the manufacturer's system. The inverter is in Hungary and hasn't sent before to the FEAK system.

In the case of measurement data, the start event is a new measurement data, that is connected to a hungarian inverter and hasn't sent before.

6.3 Tech stack

6.3.1 JSON

All data transmitted through the API is encoded in JSON format. JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for humans to read and write, and easy for machines to parse and generate. JSON represents data as key-value pairs, similar to objects in JavaScript. It supports data types such as strings, numbers, arrays, booleans, and null.

6.3.2 OCI proxy

Oracle Content and Experience Cloud includes a proxy service, so that you can use REST services which have Cross-Origin Resource Sharing (CORS) limitations or require service account credentials. The proxy service is a reverse proxy server. It provides a URL to which web browsers connect. The proxy service then acts as an intermediary between the web browser and a remote REST service (or endpoint). The proxy service explicitly adds CORS support to all endpoints and can optionally insert service account credentials to requests coming from web browsers.

6.3.3 mTLS

Mutual TLS, or mTLS for short, is a method for mutual authentication. mTLS ensures that the parties at each end of a network connection are who they claim to be by verifying that they both have the correct private key. The information within their respective TLS certificates provides additional verification. Transport Layer Security (TLS) is an encryption protocol in wide use on the Internet. TLS, which was formerly called SSL, authenticates the server in a client-server connection and encrypts communications between client and server so that external parties cannot spy on the communications.

6.3.4 RESTful API

The FEAK system exposes a RESTful API to enable standardized communication between client and server components. This API is designed following REST (Representational State Transfer) principles, ensuring stateless interactions, resource-oriented endpoints, and support for standard HTTP methods.

6.3.5 Rabbit MQ

RabbitMQ is an open-source message-broker software (sometimes called message-oriented middleware) that originally implemented the Advanced Message Queuing Protocol (AMQP) and has since been extended with a plug-in architecture to support Streaming Text Oriented Messaging Protocol (STOMP), MQ Telemetry Transport (MQTT), and other protocols. Rabbit MQ is used for receiving messages/data from the Manufacturers and to provide a queue for the erred transactions with its details.

6.4 Authentication

All access to the system is protected using certificate-based authentication to ensure secure and verified communication between clients and services.

Authentication is performed using client-side digital certificates. Each manufacturer or single device must present a valid ECC-based X.509 certificate when attempting to establish a connection. These certificates are used to verify the identity of the connecting part and to enforce access control policies.

ECC SSL certificates are the certificates that use elliptical curve cryptography algorithm to encrypt and decrypt the data transferred between a client (web browser) and web server. It's a faster and more secure encryption algorithm compared to RSA.

All certificates must be requested through FEAk Zrt., the designated certificate issuer for this system. To obtain a certificate, users must contact the security administrator of FEAk Zrt. at the following contact details: inverterhelp@feak.hu

6.4.1 Certificate request

Before a digital certificate can be issued, the requesting entity must generate a Certificate Signing Request (CSR). The CSR is a standardized, encoded file that includes the requester's public key and a set of identifying attributes, such as the Common Name (CN), Organization (O), Country (C), and other required subject fields.

As part of the CSR generation process, a cryptographic key pair is created. The private key is securely stored by the requester and must never be shared or transmitted. The corresponding public key is embedded into the CSR and is signed using the private key, serving as proof of possession.

6.4.1.1 Request flow

1. step: Fill in the lines below and save the file as {brand}_{environment}.conf, replacing brand with your specific brand name and environment with the environment (sandbox/prod) for which access is being requested.

```
[ req ]
prompt          = yes
days           = 365
```

```
distinguished_name      = req_distinguished_name
req_extensions           = v3_req

[ req_distinguished_name ]
commonName=
countryName="HU"
countryName_default="HU"
organizationName="FEAK-ENAP"
organizationName_default="FEAK-ENAP"
organizationalUnitName="Inverter manufacturer"
organizationalUnitName_default="Inverter manufacturer"
localityName="B,C"
localityName_default="B,C"
stateOrProvinceName=
emailAddress=

[ v3_req ]
extendedKeyUsage         = clientAuth
keyUsage                 = digitalSignature,
keyEncipherment
subjectKeyIdentifier     = hash
```

Except for the commonName, stateOrProvinceName and emailAddress fields, all other fields in the configuration file are fixed.

The stateOrProvinceName field indicates the geographic location from which the data is sent (i.e. the source location of the data service)

Important: The value specified in the **commonName** field must exactly match the value of the **brand** field provided in the messages during data transmission.

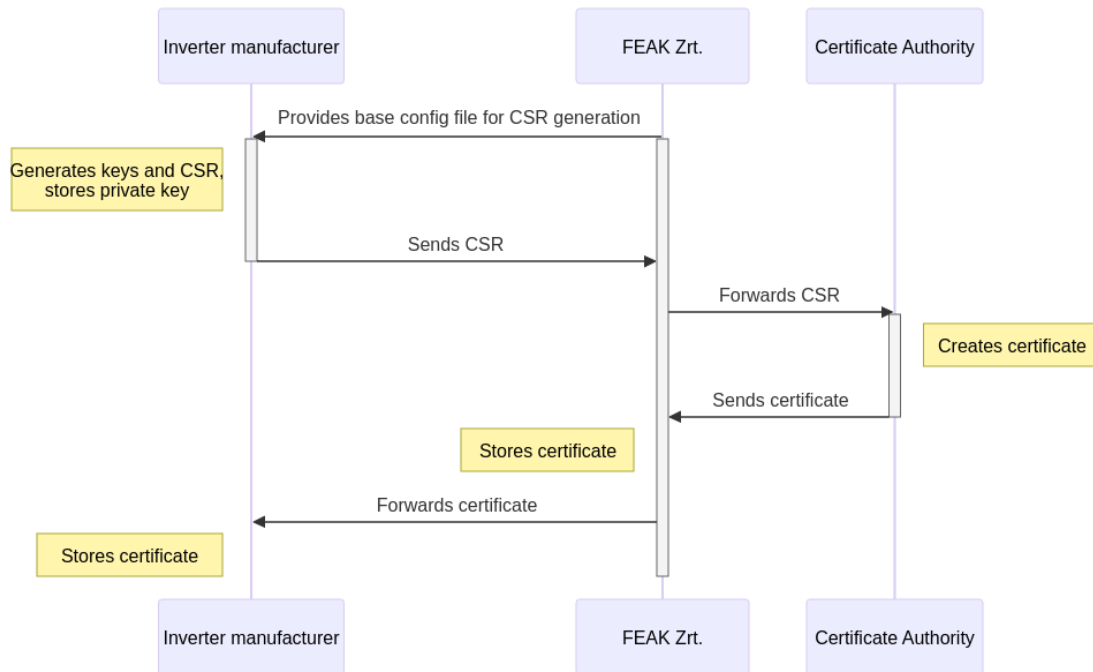
2. step: Run the following command in the folder of the saved config file.

```
openssl req -new -nodes -newkey ec:<(openssl ecparam -name
prime256v1) -keyout cert.key -out {brand}_{environment}.csr -
config {brand}_{environment}.conf
```

3. step: Forward the generated {brand}_{environment}.csr to the FEAk Zrt.

inverterhelp@feak.hu

4. step: Store and setup the returned signed certificate



3. figure - Certificate request flow

6.4.1.2 Optional commands

1. convert from .pem format to .crt

```
openssl x509 -outform der -in manufacturer.pem -out manufacturer.crt
```

2. convert from .pfx to .pem

```
openssl pkcs12 -in manufacturer.pfx -out test.pem -nodes
```

3. convert from .pfx to .crt and .key

```
openssl pkcs12 -in manufacturer.pfx -nocerts -out manufacturer.key -nodes
```

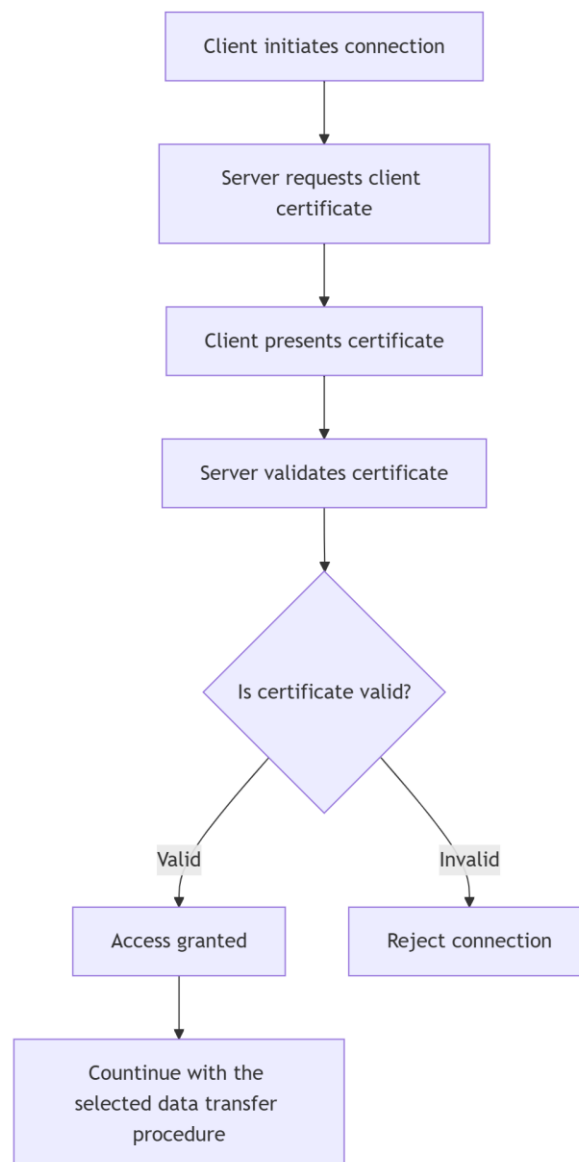
```
openssl pkcs12 -in manufacturer.pfx -nokeys -out manufacturer.crt -nodes -clcerts
```

6.4.2 Certificate testing

To test the authentication, use the following command:

```
curl -kv https://device-data-receiver.inverter-  
sandbox.feak.hu/api/v1/Versions/false --cert manufacturer.crt --  
key manufacturer.key
```

6.4.3 Authentication flow



4. figure - Authentication process

6.5 OPTION 1: Data transfer via FEAK API

API-based data ingestion enables direct, synchronous communication between systems through HTTP requests. This approach is well-suited for real-time or near-real-time data transmission, offering greater control and immediate feedback upon successful or failed data delivery.

6.5.1 Versioning

The API version number, included in the URL (e.g., /v1/), represents the specific version of the interface exposed by the system. Versioning ensures backward compatibility, allowing dependent systems to continue functioning without disruption. Additionally, it provides a clear and consistent way to communicate upcoming releases and planned changes in the API lifecycle.

6.5.2 Device-related data transfer

Inverter's manufacturers can send device-related data to the FEAK system with the following endpoint.

API endpoint:

```
POST /api/v1.1/inverter-brand/master-data
```

Request body:

```
{
  "devices": [
    {
      "serialNumber": "13548965",
      "address": {
        "country": "HU",
        "zipCode": "8956",
        "city": "ExampleCity",
        "street": "Example",
        "streetType": "utca",
        "streetCode": "10",
        "streetCodeSupplement": "",
        "building": "",

```

```
        "stairway": "",
        "door": "",
        "floor": "",
        "latitude": "46.530936",
        "longitude": "18.814042",
        "landRegistryNumber": "123/A"
    },
    "brand": "LANDIS",
    "model": "E550",
    "nominalPower": "5000",
    "acVoltageMin": "170",
    "acVoltageMax": "270",
    "installationDate": "2024-10-13",
    "removalDate": "2025-03-02"
}
]
```

Field explanation:

- **serialNumber:** Serial Number of the inverter
string, **required**
- **brand:** The manufacturer of the device. Ensure that the value is identical to the CN field
specified in [config_req.conf](#)
string, **required**
- **model:** The type and specific version of the device within the brand's product range
string, **required**
- **nominalPower:** The rated power output of the device under normal operating
conditions, in W.
integer, **required**
- **acVoltageMin:** Minimum value of the AC side voltage range, in V
integer, **required**
- **acVoltageMax:** Maximum value of the AC side voltage range, in V
integer, **required**

- **installationDate:** Date of device installation, in YYYY-MM-DD format, **required**
- **removalDate:** The date the device was removed, formatted as YYYY-MM-DD. Optional.
- **address:** location object, optional
 - **country:** 2-character ISO 3166 code for the country
String, optional (default: HU)
 - **zipCode:** Postal code of the address.
String, **required**
 - **city:** Name of the city.
String, **required**
 - **landRegistryNumber:** Identifier used in the land registry, also known as the parcel number. String, optional
 - **street:** Name of the street.
String, **required**
 - **streetType:** Type of the street (e.g., Street, Avenue, Boulevard). The value set is available in the attachment.
String, optional
 - **streetCode:** Official street code
String, optional
 - **streetCodeSupplement:** Additional code or suffix for the street, used for more precise identification. String, optional
 - **building:** Building number or identifier. String, optional
 - **stairway:** Stairway or block identifier within a building. String, optional
 - **door:** Door or apartment number. String, optional
 - **floor:** Floor number or level within the building. String, optional
 - **latitude:** Geographic latitude coordinate of the address. String, optional
 - **longitude:** Geographic longitude coordinate of the address. String, optional

Optional JSON keys are not required to be included in the request.

Within the master data, certain fields of the Address object are mandatory.

Required fields: zipcode, city, street

If the exact address information is **not available** at the manufacturer's side, the following **default values** may be used to fulfill the mandatory field requirements:

- zipcode: "0000"
- city: "unknown"
- street: "unknown"

All other mandatory master data fields (e.g. brand, model, nominal power) must still be populated according to the specification.

Response:

The API uses the following standard HTTP codes in response:

HTTP code	Meaning
200	Data processed succesfully
400	Data validation error
500	Internal server error

Limitation:

The array can contain a maximum of 1.000 devices. Any additional entries beyond this limit will not be processed.

6.5.3 Measurement data transfer

Measurement data may be obtained either directly from the inverter or provided by the manufacturers. The API exposes two separate endpoints for the submission of measurement data, based on the sender entity. The data structure and process are the same, the differences are the certificate and the endpoint.

API endpoint:

Request directly from inverters:

```
POST /api/v1.1/inverter /measurement-data
```

Request from inverter manufacturers:

```
POST /api/v1.1/inverter-brand/measurement-data
```

Request body:

```
{
  "serialNumber": "21231e32132eqw21312312",
  "readOutTs": "2024-11-18T15:23:44Z",
  "obisCode": "9.7.0",
  "brand": "fakesolar",
  "data": [
```

```
{
  {
    "timestamp": "2024-11-18T15:23:44Z",
    "value": "156"
  },
  {
    "timestamp": "2024-11-18T15:23:44Z",
    "value": "154"
  }
]
```

Field explanation:

- **messageId:** Unique identifier for the message. Version 4 UUID.
string, **required**
- **serialNumber:** Serial Number of the inverter
string, **required**
- **readOutTs:** The timestamp of when the data was read out for the device, in ISO 8601 format. **required**
- **obisCode:** The Obis code assigned by the measurement data
string, **required**
- **brand:** The manufacturer of the device. Ensure that the value is identical to the CN field specified in [config_req.conf](#)
string, **required**
- **data:** An object containing the measurement data with the following values:
 - **timestamp:** The timestamp of when the data was recorded, in ISO 8601 format.
required
 - **value:** The actual value of the measurement data, measured by the inverter.
string, **required**
Separator: Decimal point
Precision: 4 decimal places
Formatting rule: Values must be truncated after the fourth decimal place without rounding.

Response:

The API uses the following standard HTTP codes in responses:

HTTP code	Meaning/message
200	Data processed succesfully
202	Missing device data for serial number {serialNumber}
400	Data validation error
500	Internal server error

Response body for HTTP 202:

```
{
  "errorMessage": "Missing device data for serial number
{serialNumber}"
}
```

The system does not accept invalid, or incomplete data. In a case of an error, data must be resubmitted. Resubmission must include the full dataset, not only the corrected fields.

In case of HTTP code 202, the system stores the submitted measurement data, therefore, resubmission is not required. This error code indicates that the associated device-related data has not yet been registered in the system. Possible reasons are the following:

- the device-related data has not been submitted, or
- the device-related data is in a pending processing state, or
- in edge cases, the internal data service module may be temporarily unavailable (e.g., during maintenance)

The processing of device-related data may take up to 15 minutes. It is recommended to wait this period before (re)submitting the device-related data.

Limitation:

In theory, the array can contain up to 10.000 measurement records. Any additional entries beyond this limit will not be processed. In practice, since the 5-minute measurement data must be submitted at least once per hour, even in the worst-case scenario, the expected number of

records per submission is 12. Please note, the long-term goal is to achieve a near real-time data transmission.

6.5.4 Testing

The initial phase of the testing process is the transfer of [device-related](#) and [measurement data](#), as described in the previous chapters. The HTTP response codes serve as definitive indicators of transmission outcomes. A 200 status denotes successful processing, while 500 codes indicate client- or server-side errors.

For a complete end-to-end testing use the internal module's endpoints:

- [Device-related data storing](#)
- [Measurement data storing](#)

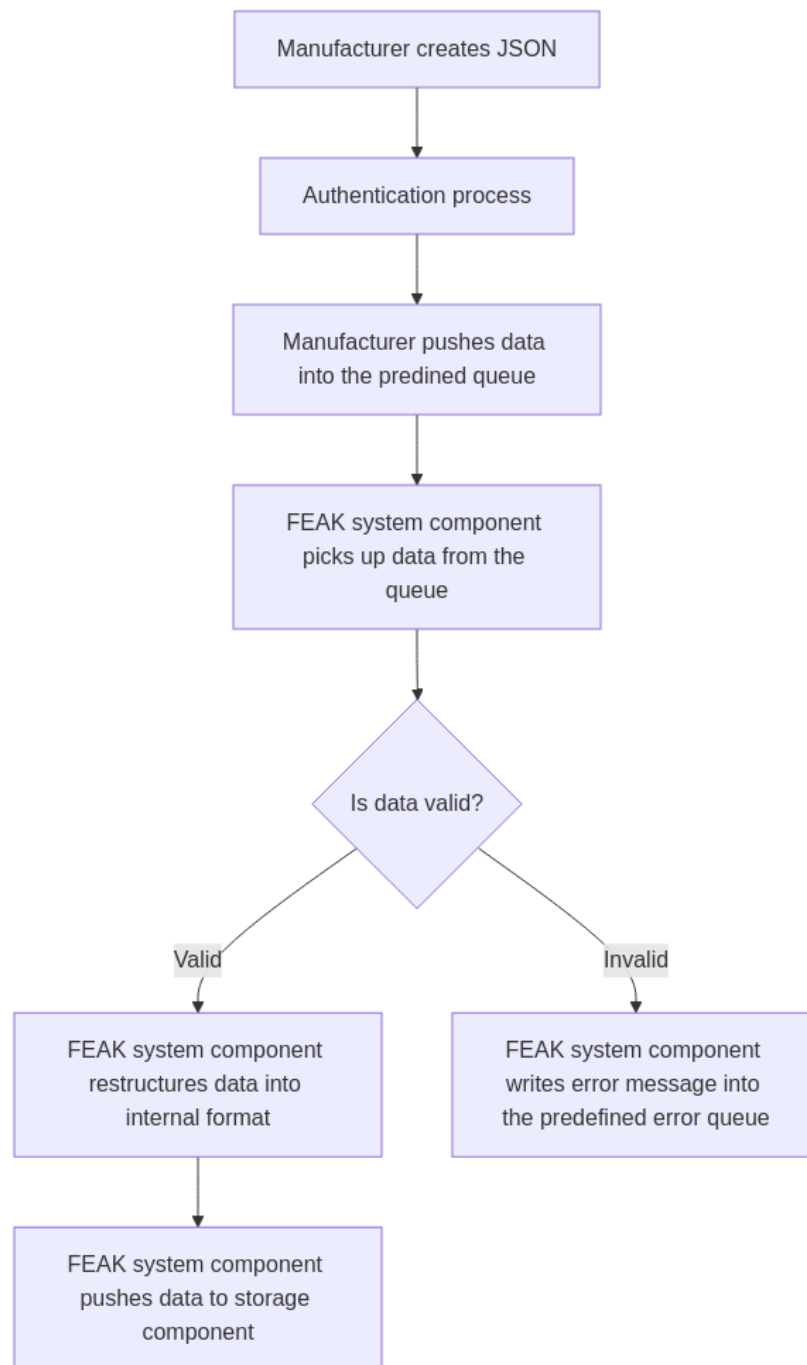
6.6 OPTION 2: Queue-based data transfer

Queue-based data transfer is an asynchronous communication model where data is pushed into a message queue, allowing the receiving system to process messages at its own pace. This method provides high reliability and decoupling between the data provider and the consumer, making it suitable for high-volume or time-insensitive data streams.

The system using a dedicated queue for device-related and measurement data to ensures that both data are processed in parallel without causing delays or conflicts, thus optimizing the overall data flow and improving the accuracy and traceability of the information.

6.6.1 Flow chart

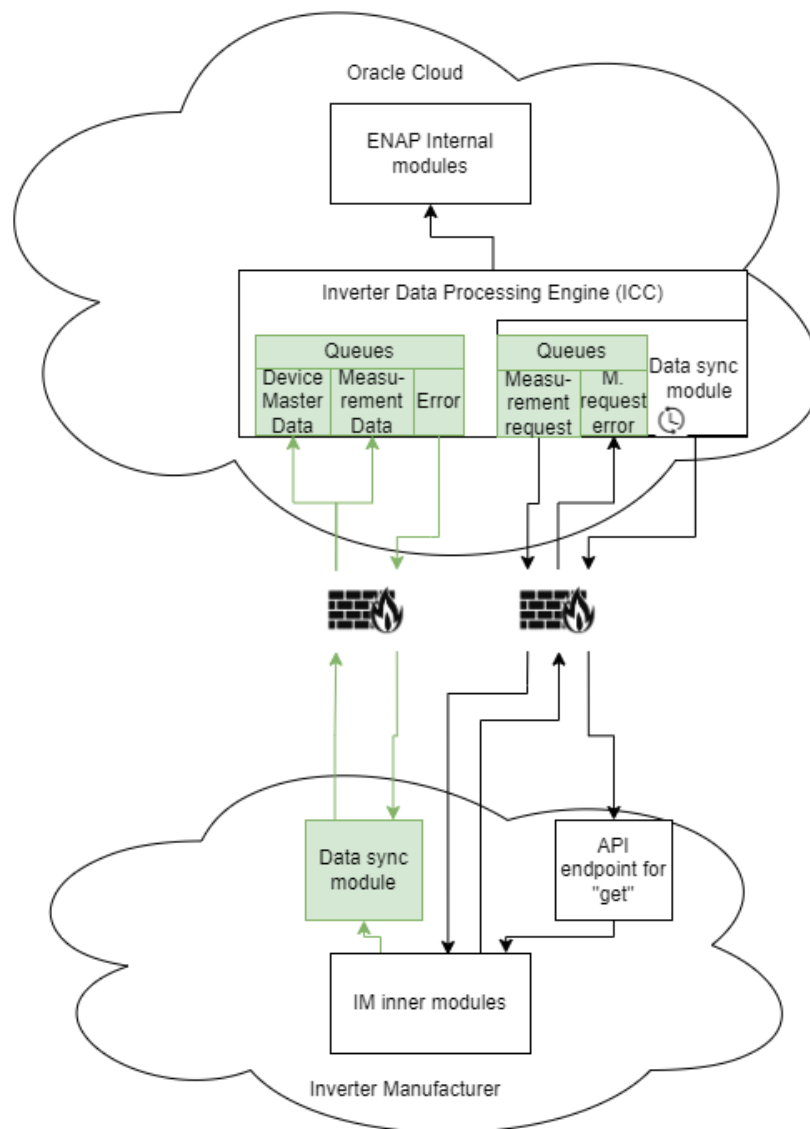
The following chart presents a high-level view of the overall process.



5. figure - Queue-based data transfer flow chart

6.6.2 Architecural overview

Below an architecture overview can be found, giving a high-level viewpoint about the components and integrations. The arrows indicate the data flow directions.



6. figure - Logical architecture of queue-based data transfer

6.6.3 Device-related data

Rabbit MQ routing strategy

- Exchange: <BRAND>.master-data
- Exchange type: direct
- Routing key: <BRAND>

The following example illustrates a fictional inverter brand called 'inverterbrand':

- Exchange: inverterbrand.master-data
- Routing key: inverterbrand

Message format

```
{
  "messageId": "67231151-ea92-4efe-a0a9-e7604669dc09",
  "serialNumber": "13548965",
  "address": {
    "country": "HU",
    "zipCode": "8956",
    "city": "ExampleCity",
    "street": "Example",
    "streetType": "utca",
    "streetCode": "10",
    "streetCodeSupplement": "",
    "building": "",
    "stairway": "",
    "door": "",
    "floor": "",
    "latitude": "46.530936",
    "longitude": "18.814042",
    "landRegistryNumber": "123/A"
  },
  "brand": "LANDIS",
  "model": "E550",
  "nominalPower": 5000,
  "acVoltageMin": 180,
  "acVoltageMax": 270,
  "installationDate": "2024-10-13",
  "removalDate": "2025-01-10"
}
```

Field explanation:

- **messageId**: Unique identifier for the message. Version 4 UUID.
string, **required**

- **serialNumber:** Serial Number of the inverter
string, **required**
- **brand:** The manufacturer of the device. Ensure that the value is identical to the CN field specified in [config_req.conf](#)
string, **required**
- **model:** The type and specific version of the device within the brand's product range
string, **required**
- **nominalPower:** The rated power output of the device under normal operating conditions, in W.
integer, **required**
- **acVoltageMin:** Minimum value of the AC side voltage range, in V
integer, **required**
- **acVoltageMax:** Maximum value of the AC side voltage range, in V
integer, **required**
- **installationDate:** Date of device installation, in YYYY-MM-DD format, required
- **removalDate:** The date the device was removed, formatted as YYYY-MM-DD. Optional.
- **address:** location object, optional
 - **country:** 2-character ISO 3166 code for the country
String, optional (default: HU)
 - **zipCode:** Postal code of the address.
String, **required**
 - **city:** Name of the city.
String, **required**
 - **landRegistryNumber:** Identifier used in the land registry, also known as the parcel number. String, optional
 - **street:** Name of the street.
String, **required**
 - **streetType:** Type of the street (e.g., Street, Avenue, Boulevard). The value set is available in the attachment.
String, optional
 - **streetCode:** Official street code
String, optional
 - **streetCodeSupplement:** Additional code or suffix for the street, used for more precise identification. String, optional

- **building:** Building number or identifier. String, optional
- **stairway:** Stairway or block identifier within a building. String, optional
- **door:** Door or apartment number. String, optional
- **floor:** Floor number or level within the building. String, optional
- **latitude:** Geographic latitude coordinate of the address. String, optional
- **longitude:** Geographic longitude coordinate of the address. String, optional

Optional JSON keys are not required to be included in the request.

Within the master data, certain fields of the Address object are mandatory.

Required fields: zipcode, city, street

If the exact address information is **not available** at the manufacturer's side, the following **default values** may be used to fulfill the mandatory field requirements:

- zipcode: "0000"
- city: "unknown"
- street: "unknown"

All other mandatory master data fields (e.g. brand, model, nominal power) must still be populated according to the specification.

6.6.3.1 Testing

All device-related data is handled and stored by the internal PodRegistry module, which is responsible validating, processing and securely persisting the incoming information. To support smooth integration and ensure system reliability, during the testing phase, internal module APIs are made available to integration partners.

The initial phase of the testing process is the transfer of device-related data, as described in the previous chapter.

To validate and verify device data use the following endpoint.

API Endpoint:

Detailed host specifications are provided in Section 7.2. (PodRegistry host)

```
GET /api/devices/by-brand/ {brand} /by-no/ {serialNumber}
```

Parameters:

- **brand:** Manufacturer's brand
- **serialNumber:** Device's serial number

Response:

HTTP code	Meaning
200	Device successfully retrieved
404	Device not found

```
{
  "content": {
    "id": 0,
    "no": "string",
    "deviceType": {...},
    "isMainMeter": false,
    "dataChannels": [...]
  }
}
```

6.6.4 Measurement data

Rabbit MQ routing strategy

Exchange: <BRAND>. measurement-data

Exchange type: direct

Routing key: <BRAND>

Message format

```
{
  "messageId": "67231151-ea92-4efe-a0a9-e7604669dc09",
  "serialNumber": "21231e32132eqw21312312",
  "readOutTs": "2024-11-18T15:50:32Z",
  "obisCode": "9.7.0",
  "data": [
```

```
{
  {
    "timestamp": "2024-11-18T15:20:44Z",
    "value": "150"
  },
  {
    "timestamp": "2024-11-18T15:25:14Z",
    "value": "154"
  }
]
```

Field explanation:

- **messageId:** Unique identifier for the message. Version 4 UUID.
string, **required**
- **serialNumber:** Serial Number of the inverter
string, **required**
- **readOutTs:** The timestamp of when the data was read out for the device, in ISO 8601 format. **Required**
- **obisCode:** The Obis code assigned by the measurement data
string, **required**
- **data:** An object containing the measurement data with the following values:
 - **timestamp:** The timestamp of when the data was recorded, in ISO 8601 format.
Required
 - **value:** The actual value of the measurement data, measured by the inverter.
string, **Required**
Separator: Decimal point
Precision: 4 decimal places
Formatting rule: Values must be truncated after the fourth decimal place without rounding.

In theory, the array can contain up to 10.000 measurement records. Any additional entries beyond this limit will not be processed. In practice, since the 5-minute measurement data must be submitted at least once per hour, even in the worst-case scenario, the expected number of records per submission is 12. Please note, the long-term goal is to achieve a near real-time data transmission.

6.6.4.1 Testing

The MDAPI module is responsible for handling and storing measurement data received from devices or external systems. This includes validating incoming data, processing it according to business logic, and ensuring secure and consistent storage. To support smooth integration and ensure system reliability, during the testing phase, internal module APIs are made available to integration partners.

The initial phase of the testing process is the transfer of measurement data, as described in the previous chapter.

To validate and verify device data use the following steps.

Important: Before testing the measurement data storage function, ensure that device data has been transmitted. Otherwise, the system will store the measurement data in the error store, making it inaccessible from the API request.

1. step: Retrieve the device's data channels from the PodRegistry module.

API endpoint:

Detailed host specifications are provided in Section 7.2. (PodRegistry host)

```
GET /api/devices/by-brand/ {brand} /by-no/ {serialNumber} /data-
channels/by-obis/ {obisCode}
```

Parameters:

- **brand:** Manufacturer's brand
- **serialNumber:** Device's serial number
- **obisCode:** The OBIS (Object Identification System) code identifying the specific measured value

Response:

HTTP code	Meaning
200	Data channel successfully retrieved
404	Data channel not found

```
{
  "content": {
    "id": 0,
    "name": "string",
```

```

        "interval": 5,
        "channelKind": "REG",
        "validFrom": "2025-01-01",
        "validUntil": "2025-10-20",
        "dataChannelTypeId": 0,
        "obisCode": "9.7.0",
        "description": "string",
        "unit": "kVA"
    }
}

```

The important response fields for step 2 are the following:

- **id:** Unique identifier of the data channel
- **channelKind:** Specifies the type of data channel; possible values are TS (Time Series) or REG (Register)

2. step: Retrieve stored measurement data using the MDAPI module

API endpoint:

Detailed host specifications are provided in Section 7.2. (MDAPI host)

```
GET /api/v1/measurements/channel/dates/ {dataChannelCategory} /
{dataChannelId}?from={from}&to={to}
```

Parameters:

- **dataChannelCategory:** Must be set to TIME_SERIES or REGISTER, depending on the channelKind value retrieved in Step 1
- **dataChannelId:** The id value retrieved in Step 1
- **from:** Start of the time range (YYYY-MM-DD)
- **to:** End of the time range (YYYY-MM-DD)

Response:

HTTP code	Meaning
200	Measurements successfully retrieved
404	Measurements not found

```
{
```



```
"id": 0,  
"dataChannelId": 0,  
"timestamp": "2025-03-21T12:30:12T",  
"data": "string",  
"state": "W",  
"type": "REGISTER",  
"savedAt": "2025-03-21T12:30:12T",  
"readoutTimestamp": "2025-03-21T12:30:12T"  
}
```

6.6.5 Error handling

Messages that fail processing are returned to a dedicated error queue for the manufacturers. If the FEAK system is unable to process a message received from a manufacturer - whether it contains measurement or device-related data - the message is automatically placed into this error queue, along with details of the error.

In order to ensure reliable message delivery and proper error handling within the integration process, the sender component is required to subscribe to the designated error message queue.

If this subscription is not configured, the sender will not receive system-generated notifications about failed submissions. Consequently, any transmission errors, such as invalid message format or processing exceptions will remain undetected by the sender.

RabbitMQ queue:

<BRAND>.error

Error message format:

```
{  
  "timestamp": "2024-11-18T15:23:44Z",  
  "originalId": "67231151-ea92-4efe-a0a9-e7604669dc09",  
  "errors": [  
    {  
      "code": "102",  
      "message": "\"messageId\" is required",  
      "invalidValue": "",  
      "path": "messageId"
```

```

    },
    {
      "code": "102",
      "message": "\"value\" is required",
      "invalidValue": "",
      "path": "data[2].value"
    }
  ],
  "originalMessage": {
    ...
  }
}

```

Fields explanation:

- **timestamp:** The timestamp of when the data was recorded, in ISO 8601 format.
Required.
- **originalId:** Unique identifier of the original message. Version 4 UUID.
String, **required**
- **errors:** the reason for failed message processing
Array, **required**
 - **code:** Specific error code. string, **required**
 - **message:** Formatted error message. String, optional.
 - **invalidValue:** The invalid value from the original message. String, optional.
 - **path:** The invalid value path in the original message (json path). String, optional.
- **originalMessage:** the original message. **Required.**

6.6.5.1 Type of queue based data transfer errors:

Description	Code	Message
Invalid message format	101	"Message can not be parsed: " + error.message E.g.: Message can not be parsed: Expected property name or '}' in JSON at position 11 (line 2 column 9)

Brand values in the master data message and the name of the queue does not match	102	Brand mismatch: Queue prefix and message's brand doesn't match
Missing required field	103	{field} is required
Empty "data" array given	104	"data" must contain at least 1 items
Invalid field format	105	Invalid format, {field} must be {format}
Passing fields not included in the specification	106	{field} is not allowed
Other unexpected error	107	The message depends on the specific issue.
Missing device data	202	Missing device data for serial number {serialNumber}

The system does not accept invalid, or incomplete data (code 101 to 107). In a case of an error, the sender is required to resubmit the dataset. Partial resubmissions are not supported; therefore, the resubmitted message must include the entire dataset, including both the corrected fields and all originally valid fields.

In case of response code 202, the system stores the submitted measurement data, therefore, resubmission is not required. This code indicates that the associated device-related data has not yet been registered in the system. Possible reasons are the following:

- the device-related data has not been submitted, or
- the device-related data is in a pending processing state

The processing of device-related data may take up to 15 minutes. It is recommended to wait this period before (re)submitting the device-related data.

RabbitMQ error queue messages are retained for a maximum of 2 days (via TTL configuration). After this period, any unprocessed messages are automatically deleted.

6.6.6 Extended functionality

In the event of data loss or other types of system failure, a direct request for measurement data from historical periods could serve as an automated solution to collect missing data.

To implement this functionality, we recommend another queue, where FEAK system could pass earlier periods to the inverter manufacturers. Manufacturers could handle these messages and provide the missing measurement data in the standard [queue-based measurement data sync](#) way as described in the previous chapters of the document. In case certain serial numbers cannot be found by manufacturer's system, that it can be indicated in a dedicated error queue.

6.6.6.1 *Historical measurement data message format*

RabbitMQ queue:

<BRAND>.measurement-data-request

Message format:

```
{
  "messageId": "39bfc275-e90a-4caf-a39d-9a717bd3393b",
  "serialNumbers":
  [
    126543721,
    5432234,
    23453212
  ],
  "obisCodes":
  [
    "31.7.0",
    "51.7.0",
    "71.7.0"
  ],
  "start": "2024-11-18T15:23:44Z ",
  "end": "2024-11-20T15:23:44Z "
}
```

The fields in the JSON structure are the following:

- **messageId**: Unique identifier for the message. Version 4 UUID.
String, **required**
- **serialNumbers**: List of inverter's serial number
List, **required**

- **obisCodes:** List of Obis code
List, **required**
- **start:** The start of the period, in ISO 8601 format. **Required.**
- **end:** The end of the period, in ISO 8601 format. **Required.**

6.6.6.2 Error handling

The following message format is used for sending measurement data request error messages when a request for historical inverter data fails.

Rabbit MQ routing strategy

- Exchange: <BRAND>. measurement-data-request-error
- Exchange type: direct
- Routing key: <BRAND>

Message format:

```
{
  "timestamp": "2024-11-18T15:23:44Z",
  "originalId": "67231151-ea92-4efe-a0a9-e7604669dc09",
  "errorMessage": "UNKNOWN_SERIAL_NUMBER",
  "serialNumber": 126543721,
  "obisCode": "31.7.0",
  "start": "2024-11-18T15:23:44Z",
  "end": "2024-11-20T15:23:44Z"
}
```

7 Sandbox environment

FEAK provides a dedicated sandbox environment to support partners during integration testing. This environment replicates the production interfaces and workflows but operates in isolation, ensuring safe and risk-free testing.

Notes:

- The environment and the certificate are for testing purposes only; no real data is processed.
- Test data is periodically deleted to maintain environment stability.
- Performance and availability may differ from production.

7.1 Requirements from inverter manufacturer side

- **IP address whitelisting**

Manufacturers must specify and provide the IP addresses from which requests will be sent.

- **Certificate acquisition**

The inverter manufacturer requires a valid certificate to facilitate the integration process. For more information, please refer to the [Authentication section](#).

7.2 Access list

7.2.1 FEAk API host

device-data-receiver.inverter-sandbox.feak.hu

7.2.2 Queue-based data transfer access points

- Exchange for device-related data:
icc-rabbitmq.inverter-sandbox.feak.hu /<BRAND>.master-data
- Exchange for measurement data:
icc-rabbitmq.inverter-sandbox.feak.hu /<BRAND>.measurement-data
- Queue for error messages:
icc-rabbitmq.inverter-sandbox.feak.hu /<BRAND>.error
- Queue for historical measurement data request:
icc-rabbitmq.inverter-sandbox.feak.hu /<BRAND>.measurement-data-request
- Exchange for historical measurement data errors:
icc-rabbitmq.inverter-sandbox.feak.hu /<BRAND>.measurement-data-request-error

The value of a „BRAND” belongs to the inverter manufacturer (written in lower case). In order to access the message queue, a valid client certificate is required. Additionally, authentication must be performed using a username and password pair, which is also issued and managed by FEAk Zrt. For further information, please contact us:

inverterhelp@feak.hu

Integration process and testingv1.24

7.2.3 PodRegistry host

pod-registry.inverter-sandbox.feak.hu

7.2.4 MDAPI host

measurement-api.inverter-sandbox.feak.hu

8 Support, contact

inverterhelp@feak.hu